Oscillatory velocity networks for budget-constrained image flow matching

Abstract

We implement an attention-free, AKOrN-inspired velocity network for unconditional image generation and a reproducible comparison with a small DiT-style transformer and recurrent convolutional control. The design isolates backbone computation while sharing a straight conditional flow-matching path, data preparation, optimizer, and numerical sampler. The planned primary comparison uses six paired CIFAR-10 seeds, 45 minutes per model, 5,000 generated images per seed, and 32 velocity evaluations. **Primary comparison incomplete; no claim about an oscillator advantage is supported.** At report generation, 2 training/control records and 2 completed evaluations were available. This is an exploratory local-compute study, not a claim of state-of-the-art performance.

Context, motivation, and introduction

Flow matching specifies a regression target for a continuous generative transport; it does not require self-attention. This experiment asks whether a coupled-oscillator parameterization gives useful sample quality for its measured cost on a small laptop GPU. The proposed advantage is falsifiable: an oscillator-minus-transformer KID difference reliably above zero contradicts improvement in this regime. A wide interval spanning zero is inconclusive. Implementation feasibility and comparative quality are separate claims.

Background and related work

Flow Matching for Generative Modeling motivates velocity regression along conditional probability paths. TorchCFM supplies the independent linear conditional path used here. Artificial Kuramoto Oscillatory Neurons, equations 4–7, motivates vector-valued spherical dynamics and linear mixing before a norm readout. This is an independent adaptation of those published equations to an image-velocity network, not copied AKOrN repository code or a reproduction of its original experiments. Scalable Diffusion Models with Transformers motivates the time-conditioned transformer control. Kuramoto Orientation Diffusion Models is related work on a different generative process; this study changes the velocity backbone while retaining the ordinary image-space transport. We make no first-in-literature claim. KID follows the kernel-based estimator described in Demystifying MMD GANs, implemented with torch-fidelity.

Method

Domain, data, and leakage control

MNIST (28×28, one channel) is the pilot domain; CIFAR-10 (32×32, RGB) is the comparison domain. Official torchvision training/test sources are downloaded with their supported archive integrity checks. A fixed stratified 5,000-example validation reservation uses split seed 20261016. SHA-256 hashes of decoded image bytes identify exact overlap; validation images overlapping test are excluded, and training examples overlapping either reserved validation or test are removed before manifests are frozen. This audit detects exact duplicates, not near-duplicates or feature-extractor pretraining contamination. Every model uses the same surviving indices, native pixel resolution, and uint8/127.5−1 normalization. There is no augmentation,

label conditioning, pretrained generator, or curriculum. Test images never enter the optimizer; final metric results do not select hyperparameters or checkpoints.

Objective and transport

For independent Gaussian noise x_0, an image x_1, and uniform t on $[0, 1]$, train

$$x_t = (1 - t)x_0 + tx_1, \qquad \mathcal{L} = \mathbb{E} \parallel v_\theta(x_t, t) - (x_1 - x_0) \parallel_2^2 .$$

The implementation uses a mean over pixels and minibatch elements. TorchCFM with sigma zero computes the path mean and conditional velocity; explicit generators supply noise and time so architecture initialization cannot consume these streams. Generation integrates $dx/dt = v_\theta(x, t)$ from Gaussian noise at zero to an image at one, using torchdiffeq's fixed midpoint solver. Sixteen outer steps require 32 actual velocity calls. Oscillator updates occur inside each call and never advance the outer flow time. The contract is `forward(images, times) -> velocities` with identical image shape and an unbounded velocity output.

Oscillator backbone

A learned stride-four patch convolution maps pixels to width 128, with fixed two-dimensional sinusoidal positions. Continuous sinusoidal flow-time features feed a two-layer SiLU time encoder. Four blocks each evolve 32 four-dimensional hidden vectors per patch for four internal Euler updates of fixed size 0.1. A learned initial embedding is normalized and expanded deterministically on every forward call; no state is carried between ODE evaluations. Within a block, let s_i be a unit vector, c_i a time-conditioned stimulus, J a learned 3×3 convolution, and $\Omega_i = (A_i - A_i^T)/2$ a learned skew-symmetric frequency matrix. Each internal update is

$$d_i = \Omega_i s_i + (Js + c)_i - s_i \langle s_i, (Js + c)_i \rangle, \qquad s_i \leftarrow \frac{s_i + 0.1 d_i}{\parallel s_i + 0.1 d_i \parallel_2}.$$

State normalization uses epsilon 10^{-8}. The conditioning stimulus uses per-image GroupNorm and a learned time-dependent affine transformation. Natural frequencies are unconstrained parameters projected to skew symmetry at each use; no symmetry assumption is imposed on the learned convolution, so no energy-minimization guarantee is claimed. States carry between the four blocks within a call. A 1×1 convolution mixes all state channels into 128 four-dimensional vectors; their stabilized norms become 128 features. Mixing before norms is essential because the individual normalized state norms would otherwise be constant. A GroupNorm and two-layer SiLU convolutional readout add a residual scaled by 0.1. A shared LayerNorm/linear patch head reconstructs dense pixel velocities. Geometry and reductions remain float32 during mixed-precision training.

Baselines, ablations, and sensitivity interventions

The transformer shares patch encoding, positional features, time encoding, and output prediction. Four blocks use four-head self-attention, a GELU MLP, and AdaLN-Zero time-conditioned residual gates. Its feed-forward width is deterministically adjusted in multiples of eight to match the oscillator trainable count within 5%; the default RGB width resolves to 576. The recurrent convolutional control retains the oscillator scaffold, convolution, iteration count, and mixed-state readout, replacing tangent dynamics with GroupNorm of a SiLU residual update. Counts, throughput, and memory are measured rather than assumed equal.

Separate no-coupling and no-natural-frequency models are trained for seeds 101 and 102. Retained parameters start from the corresponding ONN initialization. Removing coupling also removes many parameters, so its effects are not an isolated capacity-matched causal estimate. Zero frequency removes natural rotation, not all dynamics. Inference sensitivity diagnostics permute spatial hidden states after block one while preserving each vector's norm, and compare with an identity intervention on the same checkpoints and source noise. They test sensitivity to correspondence, not whether synchronization caused quality. Untrained architectures and disjoint real-versus-real image populations provide additional controls. The inference permutation is fixed before evaluation; it is not selected to maximize degradation.

Optimization, pairing, and budget

All trainable generators start from scratch with AdamW, learning rate 0.0003, betas (0.9,0.999), weight decay 0.01, effective batch 128, microbatch 32, gradient clipping at 1.0, and EMA decay 0.999. The initial rate is a conservative Adam starting point, not a validated optimum. Accumulation weights each microbatch by its fraction of the effective batch. BF16 autocast is used on CUDA while the MSE and oscillator geometry remain float32. Sampling and held-out velocity diagnostics use float32. CPU sampling is supported. No learning-rate search or adaptive quality-based stopping is used.

Seed-derived streams separate model initialization, data ordering, flow targets, validation, and generation. Data/noise/time stream prefixes are paired across architectures, while model initialization streams are independent. Unequal throughput implies different numbers of updates and examples at equal wall time; this is part of the quality-versus-cost estimand. Model execution order is randomized within each seed group before training. Checkpoints save optimizer, EMA, data permutation/cursor, CPU/CUDA RNG states, elapsed training time, dataset hash, resolved configuration, and source history. The final budget checkpoint is evaluated, not the best intermediate checkpoint.

The allocation is at most 1.5 GPU-hours for checks/pilot, 13.5 training hours for three CIFAR architectures × six seeds, three training hours for two ablations × two seeds, and six hours for sampling/evaluation/contingency. A shared file-locked ledger reserves time before GPU allocation, refunds unused reservations only on orderly exit, and charges crashed reservations conservatively. Cooperative loop checks leave cleanup time; an independent process deadline terminates a blocked worker near reservation expiry. The total ceiling is 24 occupied GPU-hours across resumes. Initialization, validation, checkpoints, downloads within reservations, and stalled wall time are conservatively charged; training-overhead accounting can therefore exceed the nominal optimizer-time allocation and consumes contingency. We do not measure energy, GPU utilization integrals, or hardware-independent FLOPs. The local target is an RTX 5070 Laptop GPU with 8 GB VRAM; actual software/hardware records accompany every run.

Measurement and pre-specified analysis

The primary outcome is unscaled KID between 5,000 generated CIFAR images and all 10,000 official test images, with 100 subsets of size 1,000 and identical torch-fidelity Inception preprocessing and metric RNG. Generation and metric seed 20261016 is fixed across arms and training seeds. Generated floats are clamped to [−1,1], mapped to [0,255], rounded, and converted to uint8; complete unclipped tensors and clipping fractions remain available. Real images retain their original uint8 values. torch-fidelity owns resizing and feature normalization for both populations. Negative unbiased KID estimates are retained. FID is secondary and

its generated/real sample counts accompany every value. MNIST uses 128-projection pixel-space sliced Wasserstein distance only as a pilot diagnostic; it is not mislabeled as semantic image quality. Velocity MSE uses the first fixed 512 surviving validation indices with a separate, training-seed-derived supervision stream, paired across architectures. Native BLAS metric threads are bounded by the recorded CPU-thread setting (default four).

For six primary seed pairs, report each KID difference, its mean, an approximate two-sided 95% Student-t interval with five degrees of freedom, and an exact two-sided paired sign-flip p-value at alpha 0.05. The t interval assumes approximately normal independent seed differences; sign-flip inference requires sign exchangeability/symmetry under the null. Six seeds cannot establish these assumptions. KID subset SD concerns one fixed image population and is not a standard error across trained models. Intervals condition on the fixed data split, test population, generation noise, metric RNG, and hardware. The shared dataset/evaluator limits external generalization. If any declared primary pair is missing, report available observations and incompleteness without complete-study inference. Ablations, fixed-seed cost curves, and interventions are descriptive secondary analyses without multiplicity-adjusted confirmatory claims.

The pre-training power calculation used 20,000 simulated Gaussian six-difference samples, all 64 sign assignments, and seed 20261016. At standardized effect 0.8, simulated rejection probability was 0.2406 (Monte Carlo standard error 0.0030). The first study is therefore exploratory and underpowered for moderate effects. Neither sample count nor training duration is extended to obtain significance. Measured throughput, synchronized generation latency, peak allocated GPU memory, parameter counts, and validation velocity MSE supplement distribution metrics. The seed-101 quality-cost curve uses 8,16,32,64 evaluations; no best seed is selected.

Results and claims

Primary comparison incomplete; no claim about an oscillator advantage is supported.

Primary paired analysis:

```
Primary paired inference is unavailable because the study is incomplete.
```

Training and capacity

Run	Status	Parameters	Updates	Train seconds	Images/s	Validation MSE
checks/mnist-smoke	step_limit	1.28949e+06	100	33.9622	376.889	2.00869
study/pilot/mnist-onn-s101	completed	1.28949e+06	3736	1200.25	398.425	0.231511

Quality and sampling cost

Evaluation	Generated/real N	NFE	KID mean	KID subset SD	FID	Pixel SWD	ms/image
checks/mnist-smoke/validation-check	128/5000	32	—	—	—	1.10101	20.9486
study/pilot/mnist-onn-s101/evaluations/validation	512/5000	32	—	—	—	0.0714237	27.8529

KID subset SD in the table is metric subset variability, not a training-seed confidence interval. FID is reported only with its finite generated/real population sizes. A missing entry denotes an unmeasured quantity. Status `untrained` is an explicit control and `step_limit` is a development run; neither qualifies as a final 45-minute checkpoint.

Numerical checks and controls

Check	Initial MSE	Final MSE	Ratio	Passed
study/checks/fixed-batch	2.25575	0.000602648	0.000267161	1

No measurements are available yet.

Failures and compute accounting

Recorded failed attempts: 1. Cumulative charged GPU seconds: 2570.1. An active reservation, if present, is charged in advance and is not all completed work. Ledger history distinguishes successful completion, exceptions, and interrupted reservations. Individual reports retain exact errors and provenance; a later repair does not erase earlier attempts.

Discussion and implications

The experimental unit is a full training seed, and the comparison applies to these implementations under equal occupied training time. Parameter matching does not equalize operations, receptive fields, memory access, optimization trajectories, or initial conditioning. A measured advantage would motivate replication across larger models and datasets; a disadvantage would constrain this specific design rather than all oscillator networks. A feasible generator can exist even when its comparative quality is worse. Inference corruption can show depen-

dence on a representation while remaining compatible with several explanations of how that representation was learned.

Limitations

This study uses small native-resolution datasets, unconditional pixel-space generation, one GPU, a small transformer, and six seeds. It does not address text conditioning, large pretrained generators, latent diffusion, or cloud-scale performance. The 24% planning power at moderate standardized effect makes inconclusive results likely. Two-seed trained ablations have weak uncertainty resolution and coupling removal changes capacity. Fixed hyperparameters may suit architectures unequally. Equal wall time depends on this hardware/software and background load; it is not a universal compute-normalized ranking. EMA with a fixed per-update decay also spans different wall-time windows when throughputs differ. Exact-duplicate exclusion does not remove semantic near-duplicates. Inception-based metrics have representation and finite-sample limitations, and MNIST pixel SWD is especially limited as a semantic measure. Deterministic settings support repeatability on this runtime, not guaranteed bitwise equivalence across hardware or library versions. Training interruptions can lose work since the last checkpoint; the budget still charges that occupied time conservatively.

Conclusion

Primary comparison incomplete; no claim about an oscillator advantage is supported. The released interface, full artifacts, and pre-specified paired analysis support independent reproduction and inspection. Stronger claims require stronger completed evidence, not additional unplanned significance searches.

Appendix A: reproducibility and artifact map

The repository README contains installation and exact CLI examples. `uv.lock` pins dependencies; the local environment uses Python 3.12 and CUDA 12.8 PyTorch wheels for Blackwell compatibility. Each run records environment versions, Git revisions, configuration, dataset manifests, logs, optimizer/EMA/RNG state, metrics, and full generated tensors. Source data and credentials are excluded from Git and model exports. Published safetensors exports load through the same model constructor with strict key checking; optimizer checkpoints use tensor/primitive-only loading. Timestamped command transcripts preserve emitted stdout/stderr, and full tensors remain binary artifacts. There is no runtime LLM service. The experiment ledger provides a lay glossary and all individual experiment records; the source protocol distinguishes pre-specified analyses from later diagnostics.

Appendix B: methodology safeguards

This design follows the distinction between evidence and speculation emphasized by Lipton and Steinhardt, data/validation pitfalls summarized by Lones, and the NeurIPS paper checklist. Fixed-batch overfitting follows the debugging practice described in Karpathy's training recipe. These references motivate procedures; they are not empirical evidence that the proposed architecture is better. All comparisons, failures, resource limits, and distributional assumptions must be read alongside the measured results.